TEC | Tecnológico de Costa Rica

Instituto Tecnológico de Costa Rica

Escuela de Computación

Lenguajes de programación

Proyecto #2 - Gestión de Matriculas

Código de curso: IC4700

Estudiante:

Alice Arias Salazar | 2023104639

Profesor:

Allan Rodríguez Davila

Grupo: 60

Contenido

Manual de usuario y pruebas de funcionalidad 2

1. **Clonar el repositorio** 2
2. **Abrir el proyecto** 2
3. **Ejecutar el programa** 3
4. **Menú principal** 3
5. **Menú creador** 4
6. **Menú analizador** 5

Descripción del problema. 8

Diseño del programa 9

Librerías usadas 12

Análisis de resultados 13

Justificación de toma de decisiones 14

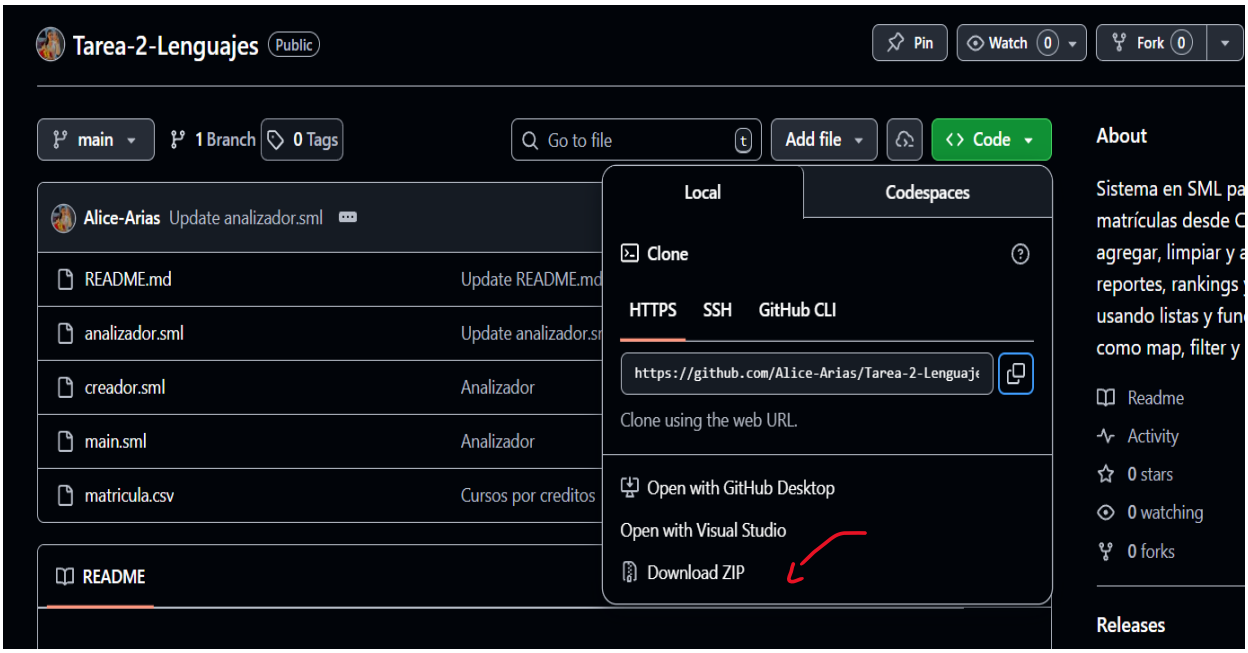
Enlace del repositorio: 15

Manual de usuario y pruebas de funcionalidad

1. Clonar el repositorio

Primero deben clonar el repositorio desde GitHub usando GitHub Desktop: https://github.com/Alice-Arias/Tarea-2-Lenguajes.git Esto descargará todos los archivos en una carpeta local en su computadora.

Figura 1



Si prefieres hacerlo de otra manera, puedes ingresar al repositorio y hacer clic en Code. Luego, como se muestra en la Figura 1, selecciona la opción Download ZIP. Una vez que la descarga se complete, descomprime el archivo y estarás listo para continuar con el siguiente paso.

2. Abrir el proyecto

Una vez descargado, abran Visual Studio Code y carguen la carpeta del proyecto vamos a abrir una terminal nueva esto para comenzar a compilar el proyecto

3. Ejecutar el programa

Figura 2

```
TERMINAL                                                    run.x86-win32
PS C:\Users\arias\OneDrive\Desktop\Tarea-2-Lenguajes\Programa> sml "main.sml"
Standard ML of New Jersey [Version 110.99.9; 32-bit; November 4, 2025]
[opening main.sml]
```

Para ejecutar el proyecto, es necesario utilizar la terminal integrada de Visual Studio Code. Primero, se debe acceder a la carpeta donde se encuentra el programa utilizando el comando **sml "main.sml",** tal como se muestra en la figura 2. Seguidamente se nos mostrara el menú principal.

4. Menú principal

Figura 3

```
PS C:\Users\arias\OneDrive\Desktop\Tarea-2-Lenguajes\Programa> sml "main.sml"

------------------
Archivo ya existe
------------------

===================================
        SISTEMA DE MATRICULA
===================================
  1. Creador
  2. Analizador
  3. Salir
-----------------------------------
Seleccione una opcion:
```

Al iniciar el programa, el sistema primero verifica si el archivo matricula.csv existe. Si el archivo no existe, lo crea automáticamente e incluye su respectivo encabezado. En cambio, si el archivo ya está presente, muestra el mensaje "Archivo ya existe" para indicar que no es necesario crearlo nuevamente. Este proceso se ejecuta antes de mostrar el menú principal, con el objetivo de evitar errores durante el uso del sistema.

Una vez realizada esta verificación, se despliega el menú donde el usuario puede elegir entre las opciones **"Creador" o "Analizador"**, las cuales se explican a continuación. En este punto, el sistema queda en espera de que el usuario ingrese una opción válida para continuar.

5. Menú creador

Figura 4

```
=====================================
     SISTEMA DE MATRICULA CREADOR
=====================================
1. Agregar estudiante
2. Limpiar catalogo
3. Volver
-----------------------------
Seleccione una opcion:
```

Al ingresar al menú de Creador, se habilita la opción 1 de agregar un estudiante, la cual permite registrar una nueva matrícula en el archivo matricula.csv. El sistema solicita primero el carnet del estudiante, el cual debe ser numérico y contener exactamente 10 dígitos; si esta condición no se cumple, no se permite continuar y se muestra un mensaje de error. Luego se solicita el nombre del estudiante, validando que no esté vacío. Después, se pide el código del curso, que debe contener 6 dígitos. A continuación, se ingresan la cantidad de créditos, que debe ser un valor numérico, y el costo por crédito. Una vez que todos los datos son ingresados correctamente y validados, el sistema registra la información y agrega al estudiante al archivo matricula.csv en la última línea. Como se muestra en la figura 5.

Figura 5

```
-----------------------------------
       Matricular Estudiante
-----------------------------------

-----------------------------------
Carnet: 2024246578

-----------------------------------
Nombre: Lupe Diaz

-----------------------------------
Curso: IC4567

-----------------------------------
Creditos: 1

-----------------------------------
Costo: 15000

-----------------------------------

Estudiante agregado correctamente
```

6. Menú analizador

Figura 6

```
========================================
Ingrese la ruta del archivo: matricula.csv

============ ANALIZADOR ============
Archivo: matricula.csv
------------------------------------
1. Cursos con mayor ingreso
2. Cursos con mas de 5 estudiantes
3. Buscar por estudiante
4. Cursos por creditos
5. Resumen general
6. Volver
------------------------------------
Seleccione:
```

Al ingresar al menú del Analizador, el sistema solicita primero la ruta del archivo. En este caso, el usuario debe ingresar la ubicación del archivo matricula.csv. Si la ruta no existe o el archivo no se encuentra, el sistema muestra un mensaje de error y no permite acceder al menú hasta que se ingrese una ruta válida. Esto asegura que el programa solo trabaje con datos correctos y disponibles como se muestra en la figura 6.

Una vez validada la ruta, se muestra el menú principal del analizador, el cual contiene diferentes opciones para procesar la información del archivo:

1. Cursos con mayor ingreso

Esta opción genera un ranking de cursos ordenados según el monto total de ingreso que producen. Para esto, el sistema solicita al usuario un valor mínimo y un valor máximo, con el fin de filtrar los resultados según el rango deseado. Luego, calcula el ingreso de cada curso multiplicando el costo del curso por la cantidad de créditos asociados, y finalmente muestra un listado ordenado de mayor a menor, indicando el código del curso y su ingreso total en colones.

Esto permite al usuario visualizar únicamente los cursos que se encuentran dentro del rango indicado y así identificar cuáles generan mayores ingresos dentro del sistema como se muestra en la figura7 , estos datos salen de la ruta que se le solicito al inicio.

Figura 7

```
=========================================
Ranking de cursos por ingreso
=========================================
Monto minimo: 1000

Monto maximo: 300000

========= Ranking de Cursos =========

Curso: IC4564 | Ingreso: CRC 210000.00
Curso: IC1234 | Ingreso: CRC 180000.00
Curso: IC4563 | Ingreso: CRC 180000.00
Curso: IC3456 | Ingreso: CRC 176000.00
Curso: IC4565 | Ingreso: CRC 154000.00
Curso: IC4567 | Ingreso: CRC 15000.00
```

2. Cursos con más de 5 estudiantes

Figura 8

```
===== Cursos con mas de 5 estudiantes =====

Curso: IC5678 | Estudiantes: 8
Curso: IC1234 | Estudiantes: 5

------------ ANALIZADOR ------------
```

En esta opción se muestran únicamente los cursos que tienen más de cinco estudiantes matriculados. El sistema cuenta la cantidad de estudiantes por curso con carnet único y filtra aquellos que cumplen esta condición, facilitando la identificación de cursos con mayor demanda como se muestra en la figura 8.

3. Buscar por estudiante

Figura 9 y 10

```
=================================================================
Buscar por carnet (exacto) o nombre del estudiante (parcial)
=================================================================

Ingrese carnet o nombre del estudiante: 2023104639

===== Resultados de Busqueda =====

Carnet: 2023104639 | Nombre: Alice Arias | Curso: IC5678 | Creditos: 4 | Costo: CRC 12000.00
Carnet: 2023104639 | Nombre: Alice Arias | Curso: IC5678 | Creditos: 4 | Costo: CRC 12000.00
Carnet: 2023104639 | Nombre: Alice Arias | Curso: IC3456 | Creditos: 1 | Costo: CRC 11000.00
```

```
=========================================
=================================================================
Buscar por carnet (exacto) o nombre del estudiante (parcial)
=================================================================

Ingrese carnet o nombre del estudiante: Al

===== Resultados de Busqueda =====

Carnet: 2023104639 | Nombre: Alice Arias | Curso: IC5678 | Creditos: 4 | Costo: CRC 12000.00
Carnet: 2023104643 | Nombre: Alana Castillo | Curso: IC1234 | Creditos: 2 | Costo: CRC 10000.00
Carnet: 2023104639 | Nombre: Alice Arias | Curso: IC5678 | Creditos: 4 | Costo: CRC 12000.00
Carnet: 2023104639 | Nombre: Alice Arias | Curso: IC3456 | Creditos: 1 | Costo: CRC 11000.00
Carnet: 2345678908 | Nombre: Alison Ramirez | Curso: IC4565 | Creditos: 2 | Costo: CRC 15400.00
```

Esta opción permite realizar búsquedas utilizando el carnet (exacto) o el nombre del estudiante (parcial) como se muestra en la figura 9 y 10. El sistema muestra todos los registros que coincidan con la búsqueda. Si no se encuentran resultados, se muestra el mensaje “No se encontraron resultados”. La búsqueda no distingue entre mayúsculas y minúsculas, lo que permite mayor flexibilidad al usuario.

4. Cursos por créditos

Figura 11

```
=========================================
Buscar cursos por la cantidad de creditos
=========================================

Ingrese la cantidad de creditos: 2

===== Resultados de Busqueda =====

Curso: IC1234
Curso: IC4565
Curso: IC1234
Curso: IC1234
Curso: IC1234
Curso: IC1234
Curso: IC4565
Curso: IC4565
Curso: IC1234
Curso: IC1234
Curso: IC1234
Curso: IC1234
Curso: IC4565
Curso: IC4565
```

En esta opción el usuario ingresa una cantidad de créditos un valor numérico y el sistema muestra los cursos que coinciden con ese valor. Si no existen cursos con esa cantidad de créditos, también se indica que no hay resultados. Esta función

permite filtrar los cursos según su carga académica como se muestra en la figura 11.

5. Resumen general

Figura 12

```
===== Resumen General =====

Estudiante con mas creditos: Daniel Rojas
Estudiante con menos creditos: Lupe Diaz
Curso con mayor ingreso: IC5678
Estudiante que mas dinero genera: Daniel Rojas
```

Esta opción muestra un resumen global del sistema, incluyendo el estudiante con más créditos, el estudiante con menos créditos, el curso con mayor ingreso y el estudiante que genera más dinero como se muestra en la figura 12.

Descripción del problema.

El sistema consiste en el desarrollo de un sistema en lenguaje Standard ML (SML) que permita gestionar y analizar información de matrículas a partir de un archivo de texto en formato CSV. El objetivo principal es aplicar los principios del paradigma funcional, utilizando listas, tuplas y funciones puras para el procesamiento de datos, así como el uso de funciones de orden superior como List.map, List.filter y List.foldl. El sistema está dividido en dos módulos principales: Creador y Analizador.

El módulo Creador permite la administración del archivo de matrículas, donde el usuario puede agregar nuevos registros ingresando datos como el carnet del estudiante, nombre, código del curso, cantidad de créditos y costo por crédito. Cada registro se almacena en el archivo respetando una estructura definida en formato CSV. Además, este módulo incluye la opción de limpiar el catálogo, la cual elimina todos los registros existentes y deja únicamente el encabezado del archivo, permitiendo reiniciar la información desde cero.

Por otro lado, el módulo Analizador se encarga de procesar la información almacenada en el archivo CSV y generar distintos análisis en tiempo real. Al iniciar, el sistema solicita la ruta del archivo a analizar y valida su existencia antes de mostrar el menú principal. Este módulo permite realizar consultas como el ranking de cursos con mayor ingreso, filtrado de cursos con más de cinco estudiantes, búsqueda de matrículas por estudiante mediante carné o nombre, filtrado de cursos según la cantidad de créditos y la generación de un resumen general del sistema con estadísticas.

Diseño del programa

El sistema fue diseñado con un enfoque funcional y modular, separando las responsabilidades en dos archivos principales: Creador y Analizador. Cada módulo se encarga de una parte específica del sistema de matrícula.

Se utilizó el archivo matricula.csv como estructura principal de almacenamiento de datos. Este archivo funciona como una base de datos simple, donde cada línea representa una matrícula con la estructura: carnet, nombre, curso, créditos y costo por crédito. El sistema valida su existencia al iniciar; si no existe, se crea automáticamente con su respectivo encabezado para evitar errores en la ejecución.

Figura 13

```
fun leerDatos () =
    let
        val carnetEstudiante = leerCarnet ()
        val nombreEstudiante = leerNombre ()
        val codigoCurso = leerCurso ()
        val cantidadCreditos = leerCreditos ()
        val costoPorCredito = leerCosto ()
    in
        (carnetEstudiante, nombreEstudiante, codigoCurso, cantidadCreditos, costoPorCredito)
    end;

(*
Objetivo: Guardar un estudiante en el archivo CSV.
Entrada: Tupla (carnet, nombre, curso, creditos, costo).
Salida: No retorna valor (unit).
Validaciones: Convierte correctamente los datos a formato string.
*)
fun agregarEstudiante (carnet, nombre, curso, creditos, costo) =
    let
        val lineaCSV =carnet ^ "," ^ nombre ^ "," ^ curso ^ "," ^Int.toString creditos ^ "," ^Real.fmt (StringCvt.FIX (SOME 2)) costo ^ "\n"

        val archivo = TextIO.openAppend "matricula.csv"
        val _ = TextIO.output(archivo, lineaCSV)
        val _ = TextIO.closeOut archivo
    in
        ()
    end;
```

Para el manejo de datos se utilizan listas y tuplas en SML, representando cada matrícula como una estructura del tipo (string * string * string * int * real) como se muestra en la figura 13.

Figura 14

```
fun leerCarnet () =
    let
        val _ = print "\n-----------------------------------\n"
        val _ = print "Carnet: "
        val entrada = limpiar (valOf (TextIO.inputLine TextIO.stdIn))
    in
        if String.size entrada = 10 andalso esNumero entrada
        then entrada
        else (print "\nError: El carnet debe tener 10 digitos numericos\n";leerCarnet ())
    end;

(*
Objetivo: Leer el código del curso.
Entrada: No recibe parámetros.
Salida: Código de curso válido (string).
Validaciones: Debe tener exactamente 6 caracteres.
*)
fun leerCurso () =
    let
        val _ = print "\n-----------------------------------\n"
        val _ = print "Curso: "
        val entrada = limpiar (valOf (TextIO.inputLine TextIO.stdIn))
    in
        if String.size entrada = 6
        then entrada
        else (print "\nError: El curso debe tener exactamente 6 caracteres\n";leerCurso ())
    end;
```

Se implementaron múltiples funciones de validación de entrada para garantizar la integridad de los datos como se muestra en la figura 14 las dos funciones de leer carnet y leer curso. Estas validaciones incluyen verificación de campos vacíos, control de longitud en el carnet y código de curso, así como validación de tipos numéricos para créditos y costos. De esta forma se evitan errores.

Para la lectura y transformación del archivo CSV se desarrolló un proceso de conversión que separa cada línea por comas y la transforma en tuplas utilizables dentro del programa. Las líneas inválidas son descartadas automáticamente para mantener consistencia en los datos.

Se hace uso intensivo de funciones de orden superior propias del paradigma funcional, como List.map, List.filter y List.foldl, las cuales permiten procesar colecciones de datos sin estructuras iterativas tradicionales. Estas funciones se utilizan para filtrar, transformar y acumular información dentro del sistema más que todo a en el módulo de Analizador.

El cálculo de ingresos por curso se realiza multiplicando la cantidad de créditos por el costo por crédito de cada matrícula, y posteriormente se acumulan los valores por curso mediante funciones. Esto permite obtener el ingreso total generado por cada curso.

Figura 15

```
(*
Objetivo: Ordenar cursos por ingreso (descendente).
Entrada: lista de cursos.
Salida: lista ordenada.
Método: quicksort.
*)
fun ordenar (listaCursos : (string * real) list) =
    case listaCursos of
        [] => []

    | (cursoBase, montoBase) :: restoCursos =>
            let
                val cursosMayores = List.filter (fn (_, monto) => monto > montoBase) restoCursos

                val cursosMenores = List.filter (fn (_, monto) => monto <= montoBase) restoCursos
            in
                ordenar cursosMayores @ [(cursoBase, montoBase)] @ ordenar cursosMenores
            end;
```

El ordenamiento de cursos por ingreso se implementa mediante un algoritmo recursivo tipo **quicksort** funcional, que separa los elementos en mayores y menores con respecto a un pivote y los reorganiza en orden descendente.

Para el análisis de estudiantes por curso se construyen estructuras que agrupan carnets por curso, evitando duplicados mediante validaciones.

Librerías usadas

Nombre	Uso	Funciones utilizadas
TextIO	Se utiliza para la lectura y escritura de archivos.	**TextIO.openIn:** abrir archivos en modo lectura. **TextIO.openOut:** crear o sobrescribir archivos. **TextIO.openAppend:** agregar datos al final del archivo. **TextIO.inputLine:** leer línea por línea. **TextIO.output:** escribir contenido en el archivo. **TextIO.closeIn / TextIO.closeOut:** cerrar archivos correctamente.
String	Manejo y procesamiento de texto.	**String.implode:** convierte lista de caracteres a string. **String.explode:** convierte string a lista de caracteres. **String.tokens:** divide cadenas según un separador. **String.size:** obtiene la longitud de un string. **String.substring:** extrae subcadenas. **String.isSubstring:** búsqueda parcial en texto.
Char	Validación de caracteres.	**Char.isDigit :** verifica si un carácter es numérico.
Int	Conversión y manejo de enteros.	**Int.fromString** : convierte string a entero
Real	Conversión y manejo de números decimales.	**Real.fromString:** convierte string a real. **Real.fromInt:** convierte entero a real. **Real.fmt:** formatea números decimales para impresión.
List	Procesamiento de listas	**List.map:** transformar elementos de una lista. **List.filter:** filtrar elementos según condición.

		List.foldl: acumular valores recorriendo la lista. **List.tl:** obtener la lista sin el primer elemento. **List.find:** buscar un elemento en la lista. **List.exists:** verificar si un elemento cumple condición. **List.mapPartial:** mapear y eliminar resultados inválidos.
StringCvt	Formateo de números para salida.	**StringCvt.FIX :** formato de decimales fijos.

Análisis de resultados

- Se desarrolló un sistema completo en Standard ML (SML) que permite registrar y analizar información de matrículas a partir de un archivo CSV, cumpliendo con el enfoque del paradigma funcional solicitado.
- Se implementó correctamente el módulo Creador, el cual permite agregar nuevos registros de matrícula con los campos solicitados: carnet de estudiante, nombre, código de curso, créditos y costo por crédito, respetando el formato establecido en el archivo.
- Se cumplió con la funcionalidad de limpieza del catálogo, permitiendo eliminar todos los registros del archivo y reiniciar el sistema dejando únicamente el encabezado.
- Se desarrolló el módulo Analizador, que solicita la ruta del archivo y permite realizar consultas en tiempo real sobre la información almacenada.
- Se implementaron las funciones de análisis requeridas:
- Ranking de cursos con mayor ingreso, con filtrado por rango mínimo y máximo.
- Identificación de cursos con más de cinco estudiantes diferentes matriculados.
- Búsqueda de matrículas por carnet exacto o nombre parcial.

- Consulta de cursos según cantidad de créditos.
- Generación de un resumen general del sistema con estadísticas globales.
- Se logró el uso correcto de estructuras funcionales como listas y tuplas para representar los datos del sistema.
- Se aplicaron funciones de orden superior (List.map, List.filter, List.foldl) en el procesamiento de información, cumpliendo con los requisitos del curso.
- Se implementó la función principal main(), permitiendo la ejecución completa del sistema desde un único punto de entrada.

Justificación de toma de decisiones

La función **List.all** se utilizó en la función **esNumero.** Se usa para verificar que todos los caracteres de una cadena cumplan una condición específica, en este caso **Char.isDigit**. Esta función se eligió porque permite validar completamente una lista de caracteres en una sola operación, asegurando que el carnet contenga únicamente números. En lugar de recorrer manualmente cada carácter por eso utilizamos **List.all**

La función **List.filter** se utilizó en varias partes del sistema, como en **limpiarCampo, buscarPorEstudiante, buscarCursosPorCreditos** y dentro de **ordenar.** Su objetivo es seleccionar únicamente los elementos que cumplen una condición. Se eligió porque facilita el filtrado de datos sin necesidad de estructuras de control. Por ejemplo, en **limpiarCampo** elimina caracteres como saltos de línea, y en **buscarPorEstudiante** permite encontrar matrículas por coincidencia de carnet o nombre.

La función **String.explode** y su complemento **String.implode** se utilizan en conjunto dentro de **esNumero** y **limpiarCampo. String.explode** convierte una cadena en lista de caracteres para poder aplicar funciones de orden superior como **List.all o List.filter**, mientras que **String.implode** reconstruye la cadena final después del procesamiento.

La función **List.foldl** es una de las más importantes del proyecto y se utiliza en funciones como: **ingresosPorCurso, estudiantesPorCurso,**

calcularCreditosPorEstudiante, calcularIngresoPorEstudiante, estudianteConMayorIngreso, estudianteConMasCreditos, estudianteConMenosCreditos y cursoConMayorIngreso. Su objetivo es recorrer una lista acumulando un resultado único. Se eligió porque permite resolver problemas de agregación como sumas, máximos o agrupaciones en una sola vez.

La función **List.map** se utilizó en **buscarCursosPorCreditos** y en transformaciones dentro de **acumulaciones**. Su propósito es aplicar una transformación a cada elemento de una lista. Se eligió porque permite extraer o modificar datos sin alterar la estructura general de la lista, lo cual es ideal cuando solo se necesita proyectar información, como obtener únicamente los cursos.

La función **List.mapPartial** se utilizó en **leerCSV** es para aplicar una transformación a cada línea del archivo y descartar automáticamente aquellas que no sean válidas. Se eligió porque permite combinar filtrado y transformación en una sola operación, lo que es especialmente útil al leer archivos donde pueden existir líneas incorrectas o mal formateadas.

La función **List.find** se utilizó en **calcularCreditosPorEstudiante y calcularIngresoPorEstudiante.** Su objetivo es buscar si un elemento ya existe dentro de una lista acumulada. Se eligió porque permite detectar duplicados de forma directa sin necesidad de implementar búsquedas manuales.

La función **String.isSubstring** se utilizó en **buscarPorEstudiante**. Su propósito es verificar si un texto está contenido dentro de otro, lo que permite búsquedas parciales por nombre. Se eligió porque brinda flexibilidad en la búsqueda, permitiendo coincidencias parciales y no solo exactas.

Enlace del repositorio: https://github.com/Alice-Arias/Tarea-2-Lenguajes.git